AROR UNIVERSITY OF ART, ARCHITECTURE, DESIGN & HERITAGE

ARTIFICIAL INTELLIGENCE LAB

LAB 05: INTRODUCTION TO NUMPY

Student Name: _____ Roll No.: _____ Date: _____

1. Lab Objectives

- Understand what NumPy is and why it is used in Python.
- Create NumPy arrays from Python lists.
- Identify array dimensions, shape, size, and data type.
- Access and update array elements using indexing and slicing.
- Perform basic arithmetic and statistical operations on arrays.
- Create arrays using built-in NumPy functions and reshape arrays.

2. Software Requirements

Python 3 and an editor such as VS Code, Jupyter Notebook, or Google Colab. NumPy must be installed.

Installation command (run in terminal if NumPy is not installed):

pip install numpy

3. Introduction and Key Definitions

NumPy	A Python library used to work efficiently with numerical data and arrays.
Array	A collection of values stored in a structured form. NumPy arrays usually contain values of the same data type.
One-dimensional (1D) array	An array containing values in a single line, similar to a list.
Two-dimensional (2D) array	An array arranged in rows and columns, similar to a table or matrix.
Indexing	Accessing an element by its position. NumPy indexing starts at 0.
Slicing	Selecting a range of elements from an array. The ending index is not included.

Shape	The number of elements along each dimension, such as (2, 3) for 2 rows and 3 columns.
Reshape	Changing an array's dimensions without changing the number of elements.
Vectorized operation	Applying an operation to all array elements without writing a loop.

4. Practical Exercises

Practical 1: Import NumPy and Create an Array

Import NumPy using the common alias np. Use np.array() to convert a Python list into a NumPy array.

```
1 import numpy as np
2
3 numbers = np.array([10, 20, 30, 40, 50])
4 print(numbers)
```

```
[10 20 30 40 50]
```

Practical 2: Create a Two-Dimensional Array

A 2D array stores values in rows and columns. Each inner list becomes one row.

```
1 import numpy as np
2
3 marks = np.array([[78, 85, 90],
4                   [66, 72, 81]])
5 print(marks)
```

```
[[78 85 90]
 [66 72 81]]
```

Practical 3: Check Array Properties

Use ndim for the number of dimensions, shape for rows and columns, size for the total number of elements, and dtype for the data type.

```python
import numpy as np

arr = np.array([[1, 2, 3], [4, 5, 6]])
print("Dimensions:", arr.ndim)
print("Shape:", arr.shape)
print("Total elements:", arr.size)
print("Data type:", arr.dtype)
```

```
Dimensions: 2
Shape: (2, 3)
Total elements: 6
Data type: int64
```

Practical 4: Indexing and Updating Elements

Use an index to access one element. In a 2D array, use [row, column]. You can also update a value.

```python
import numpy as np

arr = np.array([10, 20, 30, 40])
print(arr[0])
print(arr[2])
arr[1] = 99
print(arr)
```

```
10
30
[10 99 30 40]
```

Practical 5: Slicing an Array

Slicing selects a range. In arr[start:stop], the start is included and stop is excluded.

```python
import numpy as np

arr = np.array([5, 10, 15, 20, 25, 30])
print(arr[1:4])
print(arr[:3])
print(arr[3:])
```

```
[10 15 20]
[ 5 10 15]
[20 25 30]
```

Practical 6: Create Arrays with NumPy Functions

np.zeros() creates zeros, np.ones() creates ones, and np.arange() creates values over a range.

```python
import numpy as np

print(np.zeros(4))
print(np.ones(3))
print(np.arange(1, 6))
```

```
[0. 0. 0. 0.]
[1. 1. 1.]
[1 2 3 4 5]
```

Practical 7: Basic Arithmetic Operations

NumPy applies arithmetic element by element when arrays have compatible shapes.

```python
import numpy as np

a = np.array([2, 4, 6])
b = np.array([1, 3, 5])
print("Addition:", a + b)
print("Subtraction:", a - b)
print("Multiplication:", a * b)
print("Division:", a / b)
```

```
Addition: [ 3  7 11]
Subtraction: [1 1 1]
Multiplication: [ 2 12 30]
Division: [2.         1.33333333 1.2       ]
```

Practical 8: Useful Mathematical and Statistical Operations

NumPy provides functions such as sum(), mean(), min(), max(), and std() to summarize numeric data.

```python
import numpy as np

marks = np.array([65, 70, 80, 85, 100])
print("Sum:", np.sum(marks))
print("Mean:", np.mean(marks))
print("Minimum:", np.min(marks))
print("Maximum:", np.max(marks))
```

```
Sum: 400
Mean: 80.0
Minimum: 65
Maximum: 100
```

Practical 9: Reshape an Array

reshape() changes the layout of an array. The total number of elements must remain the same.

```python
import numpy as np

arr = np.arange(1, 7)
new_arr = arr.reshape(2, 3)
print(new_arr)
```

```
[[1 2 3]
 [4 5 6]]
```

Practical 10: Compare Values with a Condition

A comparison creates a Boolean array. You can use it to select values that satisfy a condition.

```python
import numpy as np

marks = np.array([45, 67, 82, 39, 90])
print(marks >= 50)
print(marks[marks >= 50])
```

```
[False  True  True False  True]
[67 82 90]
```

5. Quick Reference

Operation	Example
Create array	np.array([1, 2, 3])
Zeros / ones	np.zeros(3), np.ones(3)
Range of values	np.arange(1, 6)
Dimensions / shape / size	arr.ndim, arr.shape, arr.size
Index / slice	arr[0], arr[1:4]
Arithmetic	a + b, a * b
Statistics	np.mean(arr), np.max(arr)
Reshape	arr.reshape(2, 3)
Filter values	arr[arr > 10]

6. Student Tasks

Complete the following tasks independently. Write and run the code, then check your output.

A.

1. 1. Create a NumPy array containing the numbers 2, 4, 6, 8, and 10. Print the array.
2. 2. Create an array of five zeros and an array of five ones.
3. 3. Given arr = np.array([12, 24, 36, 48, 60]), print the first element, last element, and elements from index 1 to 3.
4. 4. Given marks = np.array([50, 60, 70, 80, 90]), calculate and print the sum, mean, minimum, and maximum.
5. 5. Create an array containing numbers from 1 to 12 and reshape it into 3 rows and 4 columns.

B.

6. 1. Create an array of 10 student marks: [45, 78, 92, 33, 67, 88, 51, 29, 74, 60]. Print only the passing marks (50 or above), then count how many students passed.
7. 2. Create two arrays a = np.array([3, 6, 9, 12]) and b = np.array([2, 4, 6, 8]). Calculate their element-wise sum, difference, product, and quotient.
8. 3. Create a 3 × 3 array containing numbers from 1 to 9. Print the second row, the third column, and the element in the first row and third column.
9. 4. Given temperatures = np.array([31, 34, 29, 36, 33, 38, 30]), find the average temperature, highest temperature, and all temperatures above the average.
10. 5. Create an array containing numbers from 1 to 20. Select all even numbers, square those even numbers, and print both the even numbers and their squares.

7. Lab Submission Checklist

- All code examples run without errors.
- Variable names are meaningful.
- Output is checked for each task.
- Code is properly indented and readable.

End of Lab 05